

A.

Source: World Bank Group

URL: <https://data.worldbank.org/indicator/SP.POP.TOTL>

<https://data.worldbank.org/indicator/SP.URB.TOTL>

Independent variable (x): Total Population

Dependent variable (y): Urban Population

The dataset compares the total number of people living in each country with the number of people living in urban areas. It is useful for studying the relationship between population size and urban population.

Data Set

Year 2023 World Population		
Country	Total Population	Urban Population
China	1,410,710,000	886,000,000
India	1,428,600,000	518,000,000
United States	334,900,000	273,000,000
Indonesia	277,500,000	155,000,000
Pakistan	247,500,000	91,000,000
Nigeria	227,900,000	112,000,000
Brazil	216,400,000	184,000,000
Bangladesh	171,200,000	68,000,000
Russia	143,800,000	106,000,000
Mexico	128,500,000	104,000,000
Japan	124,500,000	114,000,000
Philippines	114,891,199	52,000,000
Vietnam	99,200,000	38,000,000
Germany	83,300,000	65,000,000
Thailand	71,702,435	36,000,000

B.

```
"""Least-squares simple linear regression report generator.
```

```
Reads the first worksheet in an .xlsx file using only Python's standard  
library, fits  $y = a_0 + a_1x$ , and writes an HTML report with SVG charts.
```

```
Usage:
```

```

python lab03_sandagon.py
python lab03_sandagon.py --input dataset.xlsx --output regression_report.html
"""

from __future__ import annotations

import argparse
import html
import math
import re
import statistics
import zipfile
from pathlib import Path
from xml.etree import ElementTree as ET

NS = "{http://schemas.openxmlformats.org/spreadsheetml/2006/main}"

def read_xlsx_first_sheet(path: Path) -> list[tuple[str, float, float]]:
    """Return (label, x, y) records from columns B, C and D of worksheet 1."""
    with zipfile.ZipFile(path) as book:
        strings = []
        if "xl/sharedStrings.xml" in book.namelist():
            root = ET.fromstring(book.read("xl/sharedStrings.xml"))
            strings = ["".join(item.itertext()) for item in
root.findall(f"{NS}si")]
            root = ET.fromstring(book.read("xl/worksheets/sheet1.xml"))

        rows: list[dict[str, object]] = []
        for row in root.findall(f".://{NS}row"):
            values: dict[str, object] = {}
            for cell in row.findall(f"{NS}c"):
                match = re.match(r"([A-Z]+)", cell.attrib["r"])
                if not match:
                    continue
                column = match.group(1)
                value = cell.find(f"{NS}v")
                if value is None:
                    continue
                raw = value.text or ""
                values[column] = strings[int(raw)] if cell.attrib.get("t") == "s"
            else float(raw)
            rows.append(values)

```

```

records = []
for row in rows:
    if isinstance(row.get("B"), str) and isinstance(row.get("C"), float) and
isinstance(row.get("D"), float):
        records.append((str(row["B"]), float(row["C"]), float(row["D"])))
    if len(records) < 3:
        raise ValueError("Expected at least three numeric records in columns B
(label), C (x), and D (y).")
    return records

def svg_chart(
    points: list[tuple[float, float]], line: tuple[float, float] | None,
    title: str, x_label: str, y_label: str, zero_y: bool = False,
) -> str:
    """Create a compact SVG scatter plot; values are displayed in millions."""
    width, height = 800, 460
    left, right, top, bottom = 92, 30, 54, 72
    plot_w, plot_h = width - left - right, height - top - bottom
    xs, ys = [p[0] / 1e6 for p in points], [p[1] / 1e6 for p in points]
    x_min, x_max = min(xs), max(xs)
    y_min, y_max = min(ys), max(ys)
    if line:
        a0, a1 = line
        endpoints = [a0 / 1e6 + a1 * (x_min * 1e6) / 1e6, a0 / 1e6 + a1 * (x_max
* 1e6) / 1e6]
        y_min, y_max = min(y_min, *endpoints), max(y_max, *endpoints)
    if zero_y:
        y_min, y_max = min(y_min, 0), max(y_max, 0)
    x_pad, y_pad = max((x_max - x_min) * 0.06, 1), max((y_max - y_min) * 0.10, 1)
    x_min, x_max = x_min - x_pad, x_max + x_pad
    y_min, y_max = y_min - y_pad, y_max + y_pad

    def sx(x: float) -> float: return left + (x / 1e6 - x_min) / (x_max - x_min)
* plot_w
    def sy(y: float) -> float: return top + (y_max - y / 1e6) / (y_max - y_min) *
plot_h
    pieces = [f'<svg viewBox="0 0 {width} {height}" role="img" aria-
label="{html.escape(title)}">',
               f'<text x="{width / 2}" y="28" text-anchor="middle" class="chart-
title">{html.escape(title)}</text>']
    for i in range(6):
        x_value = x_min + i * (x_max - x_min) / 5
        y_value = y_min + i * (y_max - y_min) / 5
        x, y = left + i * plot_w / 5, top + plot_h - i * plot_h / 5

```

```

        pieces.extend([
            f'<line x1="{x:.1f}" y1="{top}" x2="{x:.1f}" y2="{top + plot_h}"
class="grid"/>',
            f'<line x1="{left}" y1="{y:.1f}" x2="{left + plot_w}" y2="{y:.1f}"
class="grid"/>',
            f'<text x="{x:.1f}" y="{top + plot_h + 22}" text-anchor="middle"
class="tick">{x_value:,.0f}</text>',
            f'<text x="{left - 10}" y="{y + 4:.1f}" text-anchor="end"
class="tick">{y_value:,.0f}</text>',
        ])
        pieces.append(f'<rect x="{left}" y="{top}" width="{plot_w}" height="{plot_h}"
class="border"/>')
        if line:
            a0, a1 = line
            pieces.append(f'<line x1="{sx(x_min * 1e6):.1f}" y1="{sy(a0 + a1 * x_min
* 1e6):.1f}" x2="{sx(x_max * 1e6):.1f}" y2="{sy(a0 + a1 * x_max * 1e6):.1f}"
class="fit"/>')
            if zero_y:
                zero = sy(0)
                if top <= zero <= top + plot_h:
                    pieces.append(f'<line x1="{left}" y1="{zero:.1f}" x2="{left +
plot_w}" y2="{zero:.1f}" class="zero"/>')
            pieces.extend(f'<circle cx="{sx(x):.1f}" cy="{sy(y):.1f}" r="4.5"
class="point"/>' for x, y in points)
            pieces.extend([
                f'<text x="{left + plot_w / 2}" y="{height - 18}" text-anchor="middle"
class="axis">{html.escape(x_label)} (millions)</text>',
                f'<text x="20" y="{top + plot_h / 2}" text-anchor="middle"
transform="rotate(-90 20 {top + plot_h / 2})" class="axis">{html.escape(y_label)}
(millions)</text>',
                '</svg>',
            ])
        return "\n".join(pieces)

def make_report(records: list[tuple[str, float, float]]) -> str:
    labels, x, y = zip(*records)
    n = len(x)
    x_bar, y_bar = statistics.fmean(x), statistics.fmean(y)
    ss_x = sum((value - x_bar) ** 2 for value in x)
    a1 = sum((xi - x_bar) * (yi - y_bar) for xi, yi in zip(x, y)) / ss_x
    a0 = y_bar - a1 * x_bar
    fitted = [a0 + a1 * xi for xi in x]
    residuals = [yi - estimate for yi, estimate in zip(y, fitted)]
    sse = sum(error ** 2 for error in residuals)

```

```

    sst = sum((yi - y_bar) ** 2 for yi in y)
    r2 = 1 - sse / sst
    syx = math.sqrt(sse / (n - 2))
    rows = "\n".join(
        f"<tr><td>{html.escape(name)}</td><td>{xi:,.0f}</td><td>{yi:,.0f}</td><td>
>{a0:,.2f} + ({a1:.8f} × {xi:,.0f}) =
<strong>{estimate:,.2f}</strong></td><td>{error:,.2f}</td></tr>"
        for name, xi, yi, estimate, error in zip(labels, x, y, fitted, residuals)
    )
    regression_chart = svg_chart(list(zip(x, y)), (a0, a1), "Urban population
versus total population", "Total population", "Urban population")
    residual_chart = svg_chart(list(zip(x, residuals)), None, "Residual plot",
"Total population", "Residual, y - ŷ", zero_y=True)
    return f"""<!doctype html>
<html lang="en"><head><meta charset="utf-8"><meta name="viewport"
content="width=device-width, initial-scale=1">
<title>Simple Linear Regression Report</title><style>
body {{ font-family: Arial, sans-serif; color: #1e293b; margin: 0; background:
#f8fafc; }} main {{ max-width: 1120px; margin: 0 auto; padding: 36px 24px 60px;
}}
h1 {{ margin: 0 0 8px; color: #0f172a; }} h2 {{ margin-top: 36px; }} .sub {{
color: #475569; }} .cards {{ display:grid; grid-template-columns:repeat(auto-
fit,minmax(190px,1fr)); gap:14px; margin:24px 0; }} .card {{ background:white;
border-radius:10px; padding:18px; box-shadow:0 1px 3px #cbd5e1; }} .label {{
color:#64748b; font-size:.88rem; }} .number {{ font-size:1.35rem; font-
weight:bold; margin-top:7px; color:#0f766e; }}
.equation {{ background:#e6fffb; border-left:4px solid #0f766e; padding:15px;
font-size:1.08rem; }} .table-wrap {{ overflow-x:auto; background:white; box-
shadow:0 1px 3px #cbd5e1; border-radius:10px; }} table {{ border-
collapse:collapse; width:100%; font-size:.88rem; }} th,td {{ padding:10px 12px;
border-bottom:1px solid #e2e8f0; text-align:right; white-space:nowrap; }}
th:first-child,td:first-child {{ text-align:left; }} th {{ background:#f1f5f9; }}
.chart {{ background:white; border-radius:10px; padding:14px; box-shadow:0 1px
3px #cbd5e1; margin:22px 0; }} svg {{ width:100%; height:auto; }} .grid {{
stroke:#e2e8f0; stroke-width:1; }} .border {{ fill:none; stroke:#94a3b8; }} .fit
{{ stroke:#e11d48; stroke-width:3; }} .point {{ fill:#0f766e; stroke:white;
stroke-width:1.5; }} .zero {{ stroke:#475569; stroke-width:1.5; stroke-
dasharray:5 4; }} .chart-title {{ font-size:17px; font-weight:bold; fill:#0f172a;
}} .tick {{ font-size:11px; fill:#475569; }} .axis {{ font-size:13px; font-
weight:bold; fill:#334155; }} footer {{ color:#64748b; font-size:.85rem; margin-
top:28px; }}
</style></head><body><main>
<h1>Simple Linear Regression by Least Squares</h1><p class="sub">n = {n}
countries · x = total population · y = urban population</p>

```

```

<div class="equation"><strong>Fitted model:</strong>  $\hat{y} = \{a0:,.2f\} + (\{a1:.8f\})x$ </div>
<div class="cards"><div class="card"><div class="label">Intercept,  $a_0$ </div><div class="number"> $\{a0:,.2f\}$ </div></div><div class="card"><div class="label">Slope,  $a_1$ </div><div class="number"> $\{a1:.8f\}$ </div></div><div class="card"><div class="label"> $S_r$  (SSE)</div><div class="number"> $\{sse:,.2f\}$ </div></div><div class="card"><div class="label"> $r^2$ </div><div class="number"> $\{r2:.6f\}$ </div></div><div class="card"><div class="label">Standard error,  $s_{y/x}$ </div><div class="number"> $\{syx:,.2f\}$ </div></div></div>
<h2>Fitted values</h2><div class="table-wrap"><table><thead><tr><th>Country</th><th>x</th><th>Observed y</th><th>Substitution:  $\hat{y} = a_0 + a_1x$ </th><th>Residual (y -  $\hat{y}$ )</th></tr></thead><tbody>\{rows\}</tbody></table></div>
<h2>Data and fitted line</h2><div class="chart">\{regression_chart\}<p class="sub">Teal points are observed data; the red line is the least-squares fitted line.</p></div>
<h2>Residuals</h2><div class="chart">\{residual_chart\}<p class="sub">The dashed horizontal line marks zero residual.</p></div>
<h2>Interpretation</h2><div class="chart"><p><strong>1. Slope and intercept.</strong> The slope,  $\{a1:.8f\}$ , means that for each additional person in a country's total population, the model predicts about  $\{a1:.3f\}$  additional urban residents. Equivalently, an increase of one million people in total population is associated with about  $\{a1 * 1e6:,.0f\}$  more urban residents. The intercept,  $\{a0:,.2f\}$  people, is the model's predicted urban population when total population is zero. Since zero population is outside the observed data range, the intercept is chiefly a mathematical starting point for the line rather than a meaningful real-world prediction.</p><p><strong>2. Fit of the line.</strong> The value  $r^2 = \{r2:.6f\}$  indicates that about  $\{r2 * 100:.1f\}$ % of the variation in urban population among these countries is explained by their total population using this linear model. The standard error,  $s_{y/x} = \{syx:,.2f\}$  people (about  $\{syx / 1e6:.1f\}$  million), is the typical vertical distance between an observed urban population and the fitted line. This indicates a strong overall linear fit, while still leaving substantial country-level differences.</p><p><strong>3. Residuals.</strong> Each residual is observed urban population minus predicted urban population: positive residuals are countries above the fitted line, and negative residuals are below it. The residual plot shows that the deviations are not all small and includes both positive and negative values. This means total population explains much of the pattern, but other factors affecting urbanization also matter; large residuals identify countries whose urban populations differ notably from the line's prediction.</p></div>
<h2>Prediction for a new x value</h2><div class="chart"><p>For a total population of <strong>x = 1,500,000,000 people</strong> (above the dataset's highest x value of 1,428,600,000 people), the predicted urban population is:</p><p class="equation"> $\hat{y} = \{a0:,.2f\} + (\{a1:.8f\} \times 1,500,000,000) = \text{<strong>\{a0 + a1 * 1\_500\_000\_000:,.2f\} people</strong>}$ </p><p>Thus, the model predicts approximately

```

```
<strong>{(a0 + a1 * 1_500_000_000) / 1e6:.2f} million urban residents</strong>.
Because this is an extrapolation beyond the observed x range, it should be
interpreted with caution.</p></div>
<footer>Computed with least squares:  $a_1 = \frac{\sum[(x_i - \bar{x})(y_i - \bar{y})]}{\sum(x_i - \bar{x})^2}$ ;  $a_0 = \bar{y} - a_1\bar{x}$ ;  $s_{y/x} = \sqrt{S_r/(n-2)}$ .</footer>
</main></body></html>"""
```

```
def main() -> None:
    parser = argparse.ArgumentParser(description="Generate a simple linear
    regression HTML report from an XLSX dataset.")
    parser.add_argument("--input", type=Path, default=Path("dataset.xlsx"),
    help="Input Excel workbook (default: dataset.xlsx)")
    parser.add_argument("--output", type=Path,
    default=Path("regression_report.html"), help="HTML report path")
    args = parser.parse_args()
    report = make_report(read_xlsx_first_sheet(args.input))
    args.output.write_text(report, encoding="utf-8")
    print(f"Wrote {args.output.resolve()}")

if __name__ == "__main__":
    main()
```

C.

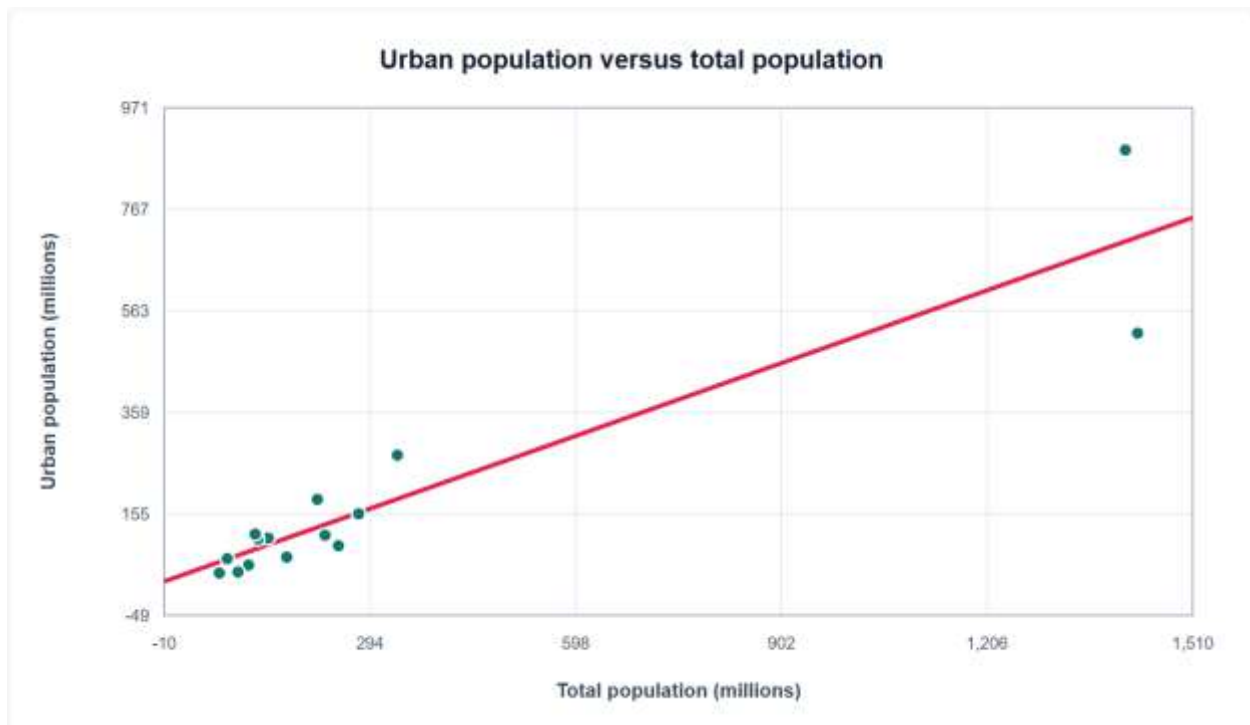
Fitted model:

$$\hat{y} = 23,900,363.96 + (0.48094571)x$$

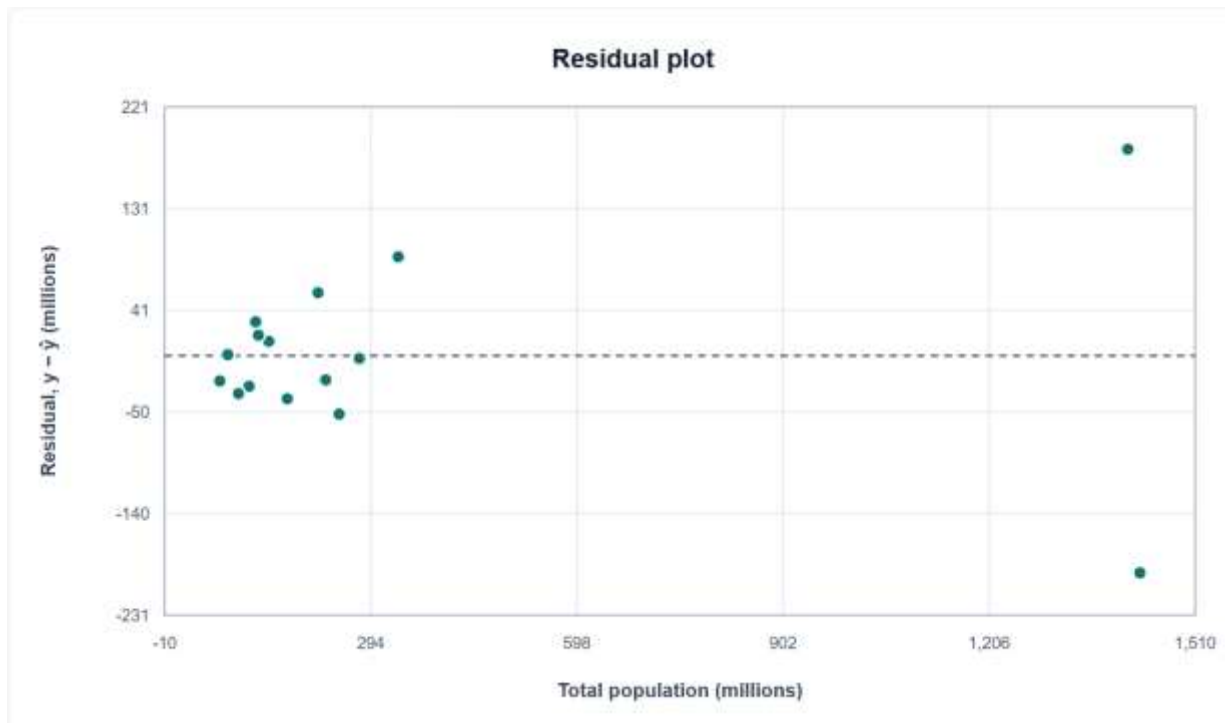
Summary of Results

a0	23,900,363.96
a1	0.48094571
S _r (SSE)	90,259,558,070,670,980
r ²	0.876776
Standard error, $s_{y/x}$	83,324,923.81

Data Points with Fitted Line



Residual Plot



Interpretation

Slope and intercept

The slope, 0.48094571, means that for each additional person in a country's total population, the model predicts about 0.481 additional urban residents. Equivalently, an increase of one million people in total population is associated with about 480,946 more urban residents. The intercept, 23,900,363.96 people, is the model's predicted urban population when total population is zero. Since zero population is outside the observed data range, the intercept is chiefly a mathematical starting point for the line rather than a meaningful real-world prediction.

Fit of the line

The value $r^2 = 0.876776$ indicates that about 87.7% of the variation in urban population among these countries is explained by their total population using this linear model. The standard error, $sy/x = 83,324,923.81$ people (about 83.3 million), is the typical vertical distance between an observed urban population and the fitted line. This indicates a strong overall linear fit, while still leaving substantial country-level differences.

Residuals

Each residual is observed urban population minus predicted urban population: positive residuals are countries above the fitted line, and negative residuals are below it. The residual plot shows that the deviations are not all small and includes both positive and negative values. This means total population explains much of the pattern, but other factors affecting urbanization also matter; large residuals identify countries whose urban populations differ notably from the line's prediction.

Prediction

For a total population of $x = 1,500,000,000$ people, the predicted urban population is:

$$\hat{y} = 23,900,363.96 + (0.48094571 \times 1,500,000,000) = 745,318,934.44 \text{ people.}$$

The prediction of **745.32 million urban residents** for a country of 1.5 billion people is meaningful for four simple reasons:

- **It stays close to observed reality:** While 1.5 billion is technically an extrapolation, it is only 5% higher than the maximum data point. That keeps it near enough to the existing large-country data to remain a plausible estimate rather than a distant guess.
- **The percentage makes demographic sense:** 745.32 million out of 1.5 billion equals 49.69%, meaning about half the population lives in cities. This sits realistically right between the two actual large countries in the sample: China with approximately 63% urbanized and India with approximately 36% urbanized.

- **Backed by a strong linear trend:** With r^2 approximately 0.88, total population explains roughly 88% of the variation in urban population across the dataset. The formula captures a reliable general tendency where bigger nations naturally have far larger urban populations.
- **Sets a macro-scale planning benchmark:** For infrastructure development and public works, a projection of roughly three-quarters of a billion city dwellers gives governments an immediate baseline scale to estimate future needs for energy, sanitation, and transit systems.